

Cours complet GSAP (GreenSock Animation Platform)

1. Installation

Via CDN (le plus simple pour débiter) :

```
<script src="https://cdnjs.cloudflare.com/ajax/libs/gsap/3.12.5/gsap.min.js"></script>
```

Via npm (projet React/Angular/Vite) :

```
npm install gsap
```

```
import { gsap } from "gsap";
```

Depuis 2024, GSAP est 100% gratuit, y compris tous les plugins premium (ScrollTrigger, SplitText, MorphSVG, etc.), car GreenSock a été racheté par Webflow.

2. Les bases : gsap.to(), gsap.from(), gsap.fromTo()

GSAP anime des propriétés CSS/JS d'un élément vers un état donné.

```
// Anime .box de son état actuel VERS x:300
gsap.to(".box", {
  x: 300,
  duration: 1,
  ease: "power2.out"
});
```

```
// Anime .box DEPUIS un état (opacité 0) VERS son état actuel
gsap.from(".box", {
  opacity: 0,
  y: -50,
  duration: 1
});
```

```
// Anime .box d'un état A précis vers un état B précis
gsap.fromTo(".box",
  { opacity: 0, x: -100 },
  { opacity: 1, x: 0, duration: 1 }
);
```

`gsap.set()` applique un état instantanément (sans animation), utile pour préparer une scène.

3. Propriétés courantes

```
gsap.to(".box", {
  x: 200,                // translateX
  y: 100,                // translateY
  rotation: 360,         // rotation en degrés
  scale: 1.5,
  opacity: 0.5,
  backgroundColor: "#ff0000",
  duration: 2,
  delay: 0.5,
  ease: "bounce.out",
  repeat: 2,             // -1 = infini
  yoyo: true,            // aller-retour
  onComplete: () => console.log("terminé !")
});
```

Easings courants : "power1" à "power4" (.in, .out, .inOut), "back", "elastic", "bounce", "none" (linéaire). Consulte greensock.com/ease-visualizer pour tester visuellement.

4. Les Timelines : enchaîner des animations

Une timeline permet de synchroniser plusieurs animations dans l'ordre, avec contrôle global (pause, play, reverse).

```
const tl = gsap.timeline({ defaults: { duration: 1, ease: "power2.out" } });

tl.to(".box1", { x: 300 })
  .to(".box2", { y: 200 }) // démarre après box1
  .to(".box3", { rotation: 180 }, "-=0.5") // chevauche de 0.5s
```

```
.to(".box4", { opacity: 0 }, "<"); // démarre EN MÊME TEMPS que l'anim p
```

Position parameters (le 3e argument) :

- "+=1" → 1s après la fin de la timeline
- "--=0.5" → 0.5s avant la fin (chevauchement)
- "<" → même moment que l'anim précédente
- ">" → juste après l'anim précédente
- "label" → à un label nommé (`tl.addLabel("scene2")`)

Contrôles : `tl.play()` , `tl.pause()` , `tl.reverse()` , `tl.seek(1.5)` , `tl.timeScale(2)` .

5. Stagger : animer plusieurs éléments en cascade

```
gsap.to(".item", {
  y: 0,
  opacity: 1,
  duration: 0.6,
  stagger: 0.15 // 0.15s de décalage entre chaque élément
});

// Stagger avancé
gsap.to(".item", {
  scale: 1,
  stagger: {
    each: 0.1,
    from: "center", // "start", "end", "center", "random", ou index
    grid: "auto"
  }
});
```

6. ScrollTrigger : animer au scroll

Le plugin le plus utilisé pour les sites modernes.

```
<script src="https://cdnjs.cloudflare.com/ajax/libs/gsap/3.12.5/ScrollTrigger.min
```

```

gsap.registerPlugin(ScrollTrigger);

gsap.to(".section", {
  x: 500,
  scrollTrigger: {
    trigger: ".section",
    start: "top center",    // quand le haut de .section touche le centre de l'écr
    end: "bottom top",
    scrub: true,           // lie l'animation à la position du scroll
    pin: false,            // épingle l'élément pendant l'animation
    markers: false         // true pour debug visuel
  }
});

```

Effet "reveal" classique au scroll :

```

gsap.utils.toArray(".reveal").forEach((el) => {
  gsap.from(el, {
    y: 80,
    opacity: 0,
    duration: 1,
    scrollTrigger: {
      trigger: el,
      start: "top 85%"
    }
  });
});

```

7. Autres plugins utiles

- **SplitText** : découpe un texte en lettres/mots/lignes pour les animer individuellement
- **Draggable** : rend un élément draggable à la souris/tactile
- **MorphSVG** : transforme une forme SVG en une autre
- **Flip** : anime des changements de layout (FLIP technique)
- **MotionPath** : anime un élément le long d'un chemin SVG

```

gsap.registerPlugin(SplitText);
const split = new SplitText(".titre", { type: "chars" });
gsap.from(split.chars, { opacity: 0, y: 20, stagger: 0.03 });

```

8. Intégration avec React (utile pour ton cas)

```
import { useEffect, useRef } from "react";
import { gsap } from "gsap";

function Box() {
  const boxRef = useRef(null);

  useEffect(() => {
    const ctx = gsap.context(() => {
      gsap.to(boxRef.current, { x: 200, duration: 1 });
    });
    return () => ctx.revert(); // nettoyage important au démontage
  }, []);

  return <div ref={boxRef} className="box" />;
}
```

`gsap.context()` isole les animations à un composant et permet un nettoyage propre — essentiel en React pour éviter les fuites mémoire.

9. Bonnes pratiques

- Anime `transform (x, y, scale, rotation)` et `opacity` plutôt que `top/left/width` → bien plus performant (GPU)
 - Utilise `will-change: transform` en CSS pour les éléments lourds à animer
 - Groupe toujours les animations liées dans une **timeline**, pas en `setTimeout`
 - `gsap.context()` en React/Vue pour éviter les animations fantômes après démontage
 - Teste tes eases sur greensock.com/ease-visualizer avant de coder
-

10. Ressources

- Documentation officielle : <https://gsap.com/docs/v3/>
- Exemples interactifs (CodePen collection GSAP) : <https://codepen.io/GreenSock>
- Forum communautaire très actif : <https://gsap.com/community/>

Tu veux que je t'écrive un exemple concret appliqué à ton portfolio React (ex: animation d'entrée des sections, ou effet au scroll) ?